CSE 4113
Introduction to Data Science
Chapter 3: Numpy & Pandas

COURSE COORDINATOR

MD. HASIBUL ISLAM
SENIOR LECTURER
CSE, NUBT KHULNA

Outlines

- Numpy: Installation
- Numpy: Array
- Numpy: ufuncs
- Pandas: Installation
- Pandas: Series
- Pandas: Read CSV
- Pandas: Read JSON

Numpy

- **NumPy**, short for **Numerical Python**
- An open-source Python library
- Supports multi-dimensional arrays (matrices)
- Provides a wide range of mathematical functions for array operations.
- Used in scientific computing, and in areas like data analysis, machine learning, etc.

Numpy: Installation

```
C:\Users\YourName>pip install numpy
```

Numpy: Array

Creating and printing Array:

```
import numpy as np

a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(a)
print(b)
print(c)
print(d)
```

Numpy

Printing how many dimensions:

```
print(a.ndim)
print(b.ndim)
print(c.ndim)
print(d.ndim)
```

Numpy: Array Indexing

Get the first element from the following array:

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr[0])
```

Print the nth element from the following array:

```
print(arr[n])
```

Access last element:

```
print(arr[-1])
```

Numpy: Array Indexing

Access the element on the first row, second column:

```
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
print('2nd element on 1st row: ', arr[0, 1])
```

Access the element on the mth row, n^{th} column:

```
print('mth element on nth row: ', arr[n, m])
```

Numpy: Array Indexing

Access the third element of the second array of the first array:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])

print(arr[0, 1, 2])
```

Numpy: Slicing

Slice elements from index 1 to index 5 from the following array:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5])
```

Slice elements from index 4 to the end of the array:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[4:])
```

Numpy: Slicing

Slice elements from the beginning to index 4 (not included):

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[:4])
```

Slice from the index 3 from the end to index 1 from the end:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[-3:-1])
```

Numpy: Slicing

Return every other element from index 1 to index 5:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5:2])
```

Return every other element from the entire array:

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[::2])
```

Numpy: Shape and Reshape

Shape:

```
import numpy as np
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(arr.shape)
```

Reshape:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(4, 3)
print(newarr)
```

Numpy: Array Iteration

Iterate on the elements of the following 1-D array:

```
import numpy as np

arr = np.array([1, 2, 3])

for x in arr:

  print(x)
```

Numpy: Array Iteration

Iterate on the elements of the following 2-D array:

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
  print(x) #Here, x indicates each 1-D array
```

Iterate on each scalar element of the 2-D array:

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
for x in arr:
  for y in x:
    print(y)
```

Numpy: Array Iteration

Iterate on the elements of the following 3-D array:

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
for x in arr:
  print(x)
```

Note: Here, x indicates each 2-D array

Iterate down to the scalars:

```
import numpy as np
arr = np.array([[[1, 2, 3], [4, 5, 6]], [[7, 8, 9], [10, 11, 12]]])
for x in arr:
  for y in x:
    for z in y:
      print(z)
```

Numpy: Array Join

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.concatenate((arr1, arr2))
print(arr)
```

Numpy: Array Join

```
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.concatenate((arr1, arr2))
print(arr)
```

Join two 2-D arrays along rows (axis=1):

```
import numpy as np
arr1 = np.array([[1, 2], [3, 4]])
arr2 = np.array([[5, 6], [7, 8]])
arr = np.concatenate((arr1, arr2), axis=1)
print(arr)
```

Numpy: Array Split

Split the array in 3 parts:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

newarr = np.array_split(arr, 3)

print(newarr)
```

Note: We also have the method `split()` available but it will not adjust the elements when elements are less in source array for splitting, `array_split()` worked properly but `split()` would fail.

Numpy: Array Split

Access the splitted arrays:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6])

newarr = np.array_split(arr, 3)

print(newarr[0])

print(newarr[1])

print(newarr[2])
```

Numpy: Array search

Find the indexes where the value is 4:

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5, 4, 4])
x = np.where(arr == 4)
print(x)
```

Find the indexes where the value 7 should be inserted:

```
import numpy as np
arr = np.array([6, 7, 8, 9])
x = np.searchsorted(arr, 7)
print(x)
```

Numpy: Array sort

Sort the array alphabetically:

```
import numpy as np
arr = np.array(['banana', 'cherry', 'apple'])
print(np.sort(arr))
```

Sort a boolean array:

```
arr = np.array([True, False, True])
print(np.sort(arr))
```

Sort a 2-D array:

```
import numpy as np
arr = np.array([[3, 2, 4], [5, 0, 1]])
print(np.sort(arr))
```

Numpy: ufuncs

Add the values in arr1 to the values in arr2:

```
import numpy as np

arr1 = np.array([10, 11, 12, 13, 14, 15])

arr2 = np.array([20, 21, 22, 23, 24, 25])

newarr = np.add(arr1, arr2)

print(newarr)
```

Numpy: ufuncs

Unary ufuncs:

Function	Description
`abs, fabs`	Compute the absolute value element-wise for integer, floating-point, or complex values
`sqrt`	Compute the square root of each element (equivalent to `arr ** 0.5`)
`square`	Compute the square of each element (equivalent to `arr ** 2`)
`exp`	Compute the exponent e^x of each element
`log, log10, log2, log1p`	Natural logarithm (base *e*), log base 10, log base 2, and log(1 + x), respectively
`sign`	Compute the sign of each element: 1 (positive), 0 (zero), or –1 (negative)
`ceil`	Compute the ceiling of each element (i.e., the smallest integer greater than or equal to that number)
`floor`	Compute the floor of each element (i.e., the largest integer less than or equal to each element)

Numpy: ufuncs

Unary ufuncs:

`rint`	Round elements to the nearest integer, preserving the `dtype`
`modf`	Return fractional and integral parts of array as separate arrays
`isnan`	Return Boolean array indicating whether each value is `NaN` (Not a Number)
`isfinite, isinf`	Return Boolean array indicating whether each element is finite (non-`inf`, non-`NaN`) or infinite, respectively
`cos, cosh, sin, sinh, tan, tanh`	Regular and hyperbolic trigonometric functions
`arccos, arccosh, arcsin, arcsinh, arctan, arctanh`	Inverse trigonometric functions
`logical_not`	Compute truth value of `not x` element-wise (equivalent to `~arr`)

Numpy: ufuncs

Binary ufuncs:

Function	Description
`add`	Add corresponding elements in arrays
`subtract`	Subtract elements in second array from first array
`multiply`	Multiply array elements
`divide, floor_divide`	Divide or floor divide (truncating the remainder)
`power`	Raise elements in first array to powers indicated in second array
`maximum, fmax`	Element-wise maximum; `fmax` ignores `NaN`
`minimum, fmin`	Element-wise minimum; `fmin` ignores `NaN`
`mod`	Element-wise modulus (remainder of division)
`copysign`	Copy sign of values in second argument to values in first argument
`greater, greater_equal, less, less_equal, equal, not_equal`	Perform element-wise comparison, yielding Boolean array (equivalent to infix operators `>, >=, <, <=, ==, !=`)
`logical_and`	Compute element-wise truth value of AND (&) logical operation
`logical_or`	Compute element-wise truth value of OR (|) logical operation
`logical_xor`	Compute element-wise truth value of XOR (^) logical operation

Numpy: ufuncs

Custom ufuncs:

```
import numpy as np
def myadd(x, y):
  return x+y

myadd = np.frompyfunc(myadd, 2, 1)
print(myadd([1, 2, 3, 4], [5, 6, 7, 8]))
```

Pandas

- Pandas is an open-source Python Library
- providing high-performance data manipulation and analysis tool using its powerful data structures
- The name Pandas is derived from the word Panel Data an Econometrics from Multidimensional data.

Pandas: Insatallation

```
C:\Users\Your Name>pip install pandas
```

Pandas: Series

A Pandas Series is like a column in a table.

It is a one-dimensional array holding data of any type.

```
import pandas as pd
a = [1, 7, 2]
myvar = pd.Series(a)
print(myvar)
```

Pandas: Series

Labels

If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc.

This label can be used to access a specified value.

Return the first value of the Series:

```
print(myvar[0])
```

Pandas: Series

Create Labels

With the `index` argument, you can name your own labels.

Create your own labels:

```
import pandas as pd
a = [1, 7, 2]
myvar = pd.Series(a, index = ["x", "y", "z"])
print(myvar)
```

Return the value of "y":

```
print(myvar["y"])
```

Pandas: Read CSV

Load the CSV into a DataFrame:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.to_string())
```

Print the DataFrame without the `to_string()` method:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df)
```

Pandas: Read CSV

Check the number of maximum returned rows:

```
import pandas as pd
print(pd.options.display.max_rows)
```

Increase the maximum number of rows to display the entire DataFrame:

```
import pandas as pd
pd.options.display.max_rows = 9999
df = pd.read_csv('data.csv')
print(df)
```

Pandas: Read JSON

Load the JSON file into a DataFrame:

```
import pandas as pd
df = pd.read_json('data.json')
print(df.to_string())
```

Pandas: Read JSON

Load a Python Dictionary into a DataFrame:

```
import pandas as pd

data = {
  "Duration":{
    "0":60,
    "1":60,
  },
  "Pulse":{
    "0":110,
    "1":117,
  },
  "Maxpulse":{
      "0":130,
      "1":145,
    },
  "Calories":{
      "0":409,
      "1":479,
      "2":340,
    }
}
df = pd.DataFrame(data)
print(df)
```



Thank you

for your Attention